

Arquitectura del Motor C++ (The Mechanic's Apprentice)

1. Gestión de Memoria: El patrón RAII

RAII significa *Resource Acquisition Is Initialization* (La adquisición del recurso es la inicialización). Es la regla de oro del C++ moderno.

- **El Problema:** En C++, si cargas algo en la tarjeta gráfica (un modelo 3D, una textura, un sonido) y te olvidas de borrarlo al terminar, creas un *Memory Leak* (Fuga de memoria). El juego consumirá cada vez más RAM hasta colapsar.
- **Nuestra Solución (RAII):** Nos aseguramos de que cada clase sea responsable de su propia basura. En el **Constructor** (`Player::Player`) cargamos el modelo usando `LoadModel()`. En el **Destructor** (`Player::~~Player`), que es una función que C++ llama automáticamente justo antes de destruir al jugador, ponemos el `UnloadModel()`.
- **Por qué importa:** Nunca tenemos que acordarnos de limpiar la memoria manualmente en medio de un nivel. Si el enemigo muere y desaparece, su destructor se ejecuta solo y la VRAM se limpia mágicamente.

2. Lógica de Entidades: Máquina de Estados Finitos (FSM)

Casi toda la lógica del juego funciona con el archivo `StateMachine.h`. Una FSM garantiza que un personaje **solo puede estar en un estado a la vez**.

- **El Problema (Spaghetti Code):** Si usáramos puros `if (estaCorriendo && !estaAtacando && !estaHurt) { ... }`, el código crecería hasta ser inmanejable y lleno de bugs (ej: atacar y hacer un dash a la vez).
- **Nuestra Solución:** Creamos cajas estancas. El jugador tiene estados: `Idle`, `Run`, `Attack`, `Dash`, `Hurt`. Cada estado tiene sus propias funciones (`UpdateIdle`, `UpdateRun`).
- **El truco:** Si estás en el estado `Dash`, el juego solo ejecuta `UpdateDash`. No lee el teclado para caminar ni te deja atacar hasta que el temporizador termine y la máquina te devuelva forzosamente al estado `Run` o `Idle`.

3. Filosofía "Data-Driven" (Orientado a Datos)

El motor de C++ es "tonto". No sabe cuántos enemigos hay, ni dónde están, ni cuánta vida tienen, hasta que lee el archivo JSON.

- **El Problema:** Si quisieras cambiar el HP del enemigo de 30 a 50, tendrías que ir al código C++, cambiar el número, y esperar a que el proyecto compile de nuevo (lo cual en proyectos grandes tarda minutos u horas).
- **Nuestra Solución:** Separamos los datos del código usando la librería **JSON** y construyendo el **Editor de Niveles en C#**.
- **Por qué es nivel AAA:** Los Game Designers pueden usar el editor C#, cambiar la posición de los obstáculos, añadir 20 engranajes más, exportar el JSON y jugar instantáneamente sin tocar ni compilar una sola línea de C++.

4. Matemáticas de Videojuegos aplicadas

Hemos usado álgebra vectorial básica para hacer que el *Game Feel* sea profesional, utilizando la librería **CollisionMath.h** que creamos.

- **Normalización de Vectores (**Normalize2D**):** Cuando el jugador pulsa 'W' (Arriba) y 'D' (Derecha) a la vez, el vector resultante es diagonal. Por el teorema de Pitágoras, la diagonal de un cuadrado es más larga que sus lados. Si no "normalizamos" (reducir la longitud del vector exactamente a 1), el jugador correría más rápido en diagonal que en línea recta.
- **Rotación Trigonométrica (**atan2f**):** En las funciones **Draw()** de los modelos, usamos el arco tangente (**atan2f(dir.x, dir.z)**). Esta función matemática toma tu vector de dirección (hacia dónde caminas) y devuelve exactamente el ángulo en el que hay que rotar el modelo 3D para que mire hacia allí.
- **Físicas de Retroceso (**Knockback**):** Cuando un enemigo te pega, no solo te resta HP. Te inyecta un vector de fuerza (**knockbackDir**). En la función global **Update(float dt)**, le sumamos esa fuerza a tu posición y la vamos reduciendo multiplicándola por un factor de fricción (Drag) hasta que te frenas.

5. Separación del "Update" y el "Draw"

Si miras el motor, verás que las físicas y cálculos siempre se hacen en **Update(float dt)**, y el renderizado siempre en **Draw()**.

- **dt (Delta Time):** Es el tiempo que ha pasado desde el frame anterior. Multiplicamos todas las velocidades por **dt** (**velocidad * dt**). Así aseguramos que el jugador se mueva a la misma velocidad sin importar si el juego va a 30 FPS o a 144 FPS.
- **Por qué importa:** Nunca debes mezclar cálculos lógicos con dibujado gráfico. Si un día decides cambiar los gráficos de 3D a 2D, solo tendrás que borrar la función **Draw()** y la lógica de físicas y colisiones (**Update**) seguirá funcionando perfectamente intacta.

Te recomiendo leer este documento un par de veces y luego ir a los archivos **Player.cpp**, **Application.cpp** o **CollisionMath.h** para ver cómo estos conceptos abstractos se han convertido en líneas de código reales.